INSPIRE AI

STAND-UP

Standup Meeting



© 2026 Inspire | Confidential

REVIEW

Review Template Database Storage

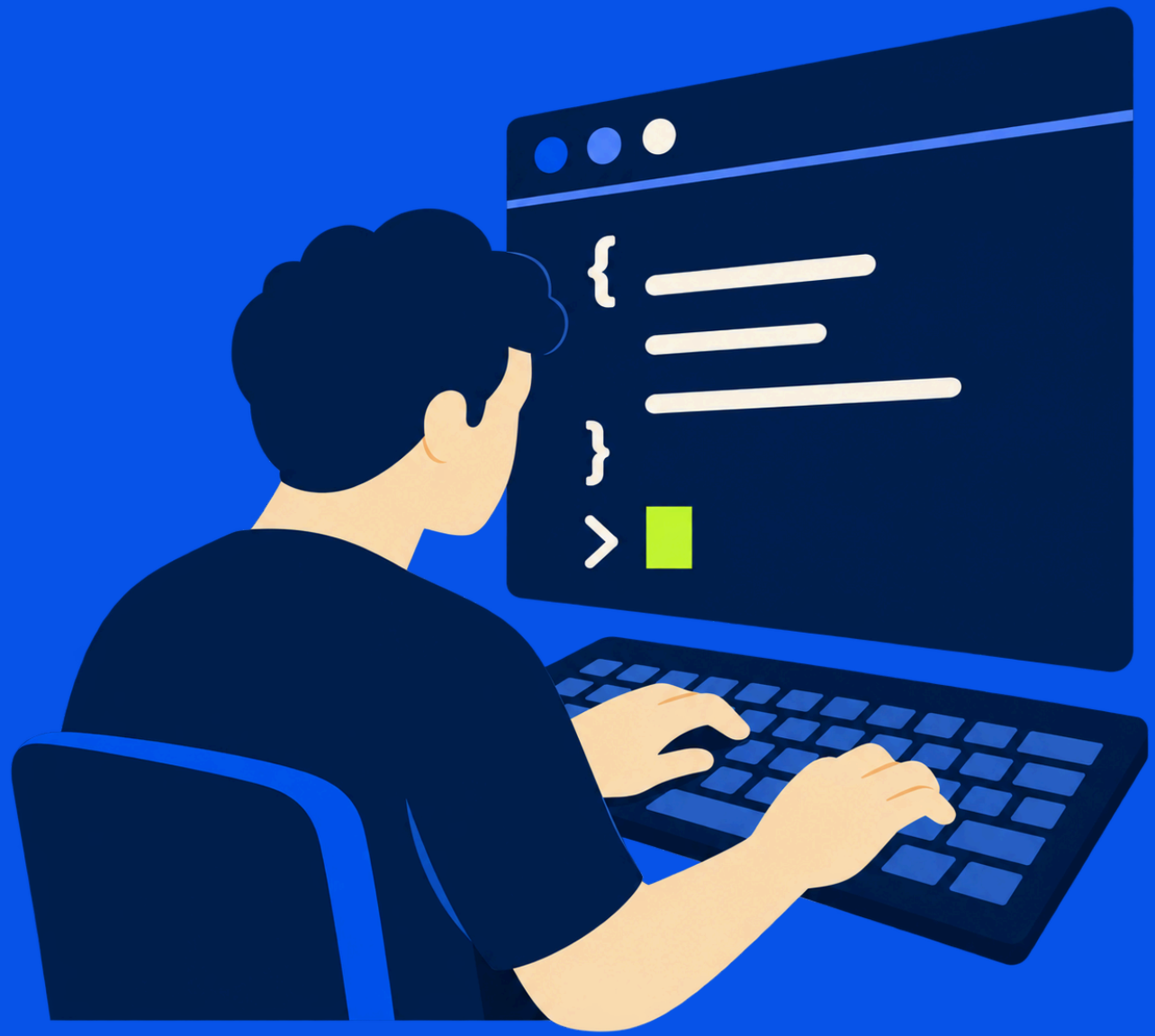


© 2026 Inspire | Confidential

INSPIRE AI

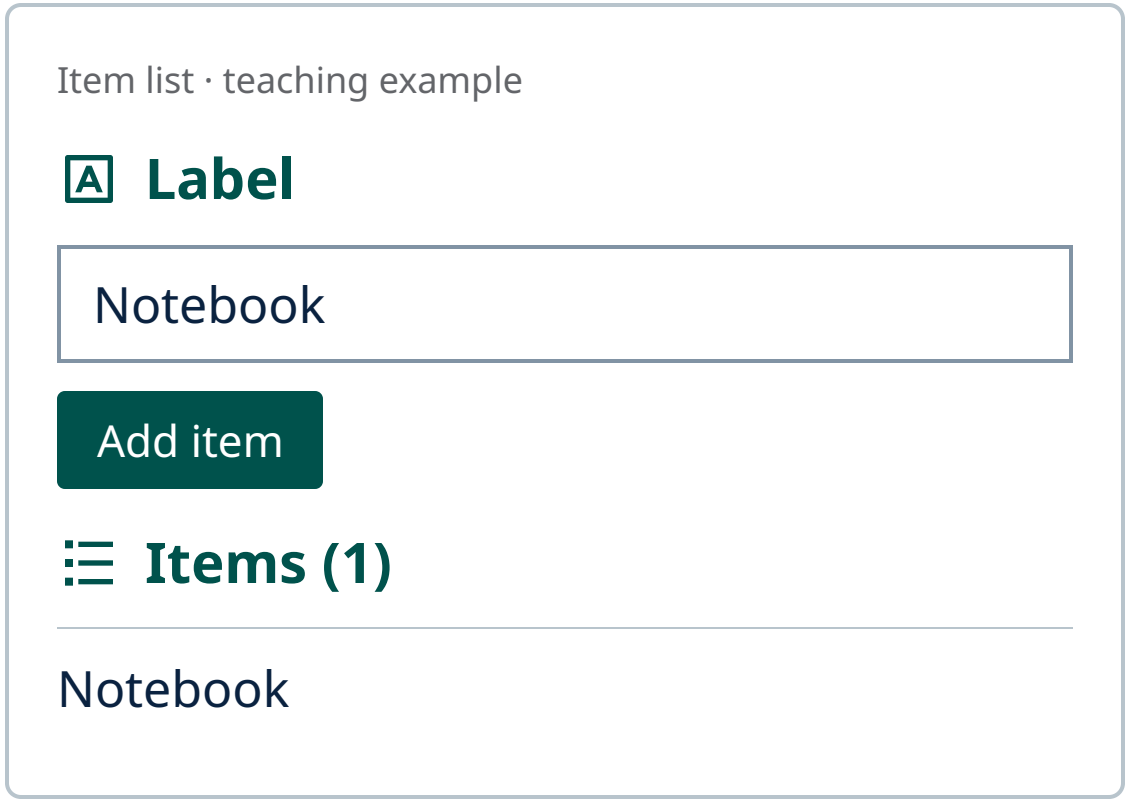
DEMO

Build Components and Pass Data



© 2026 Inspire | Confidential

One action, one visible change



Edit a draft

Typing changes the input.

Submit an item

A valid item joins the list.

Explain the result

Trace the data behind the screen.

Illustrative UI. Course tasks use the supplied parcel form and its rules.

© 2026 Inspire | Confidential

A page has structure, style and behavior

HTML
Elements and meaning
label · input · button

CSS
Appearance and layout
spacing · color · size

JavaScript
Behavior and data
type · click · submit

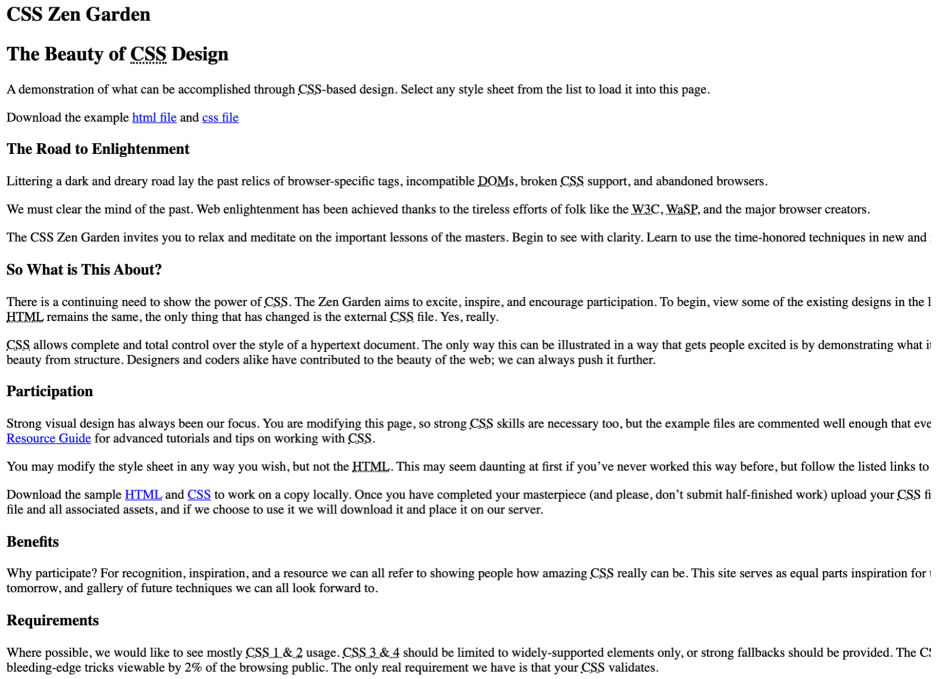
One browser page

DOM = the browser’s live tree of page nodes, not a screenshot or source file.

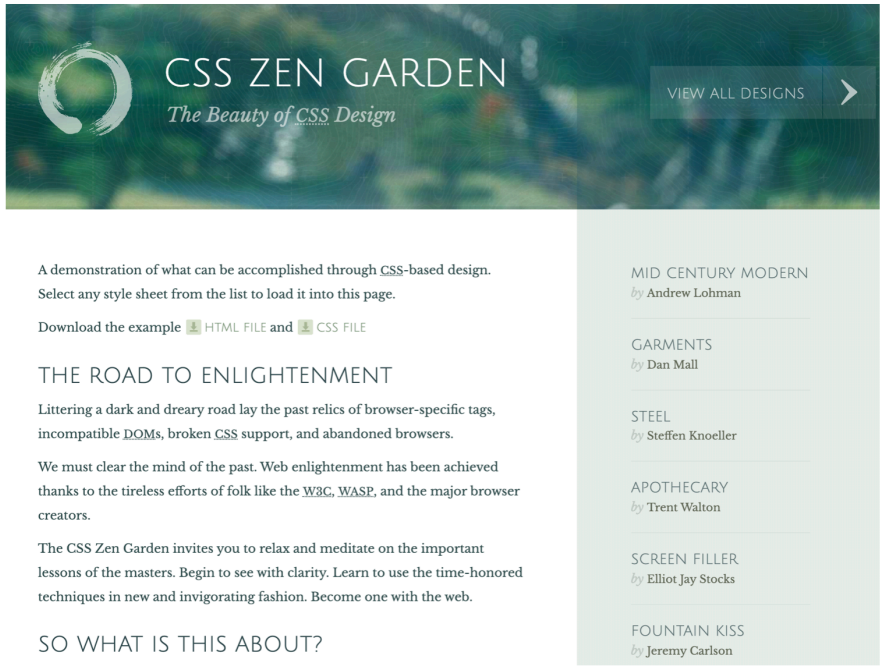
Need the HTML / JSX bridge? →

© 2026 Inspire | Confidential

Same content, different style



Without CSS



With CSS

Compare structure and appearance; no need to read the small text.

© 2026 Inspire | Confidential

Today, interaction stays in the browser

Today

Dev server → page files → browser

The browser runs the UI code.

Later

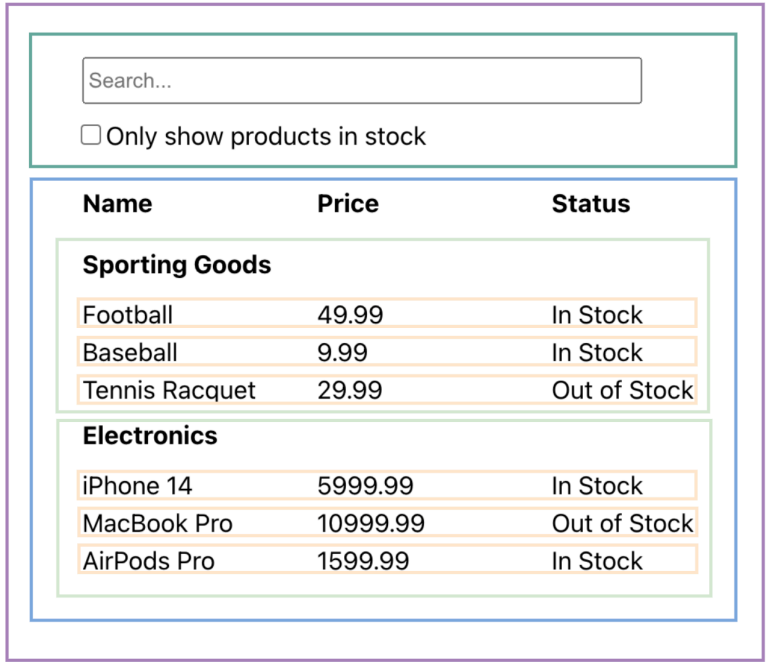
Browser → business API → database

This path is not connected yet.

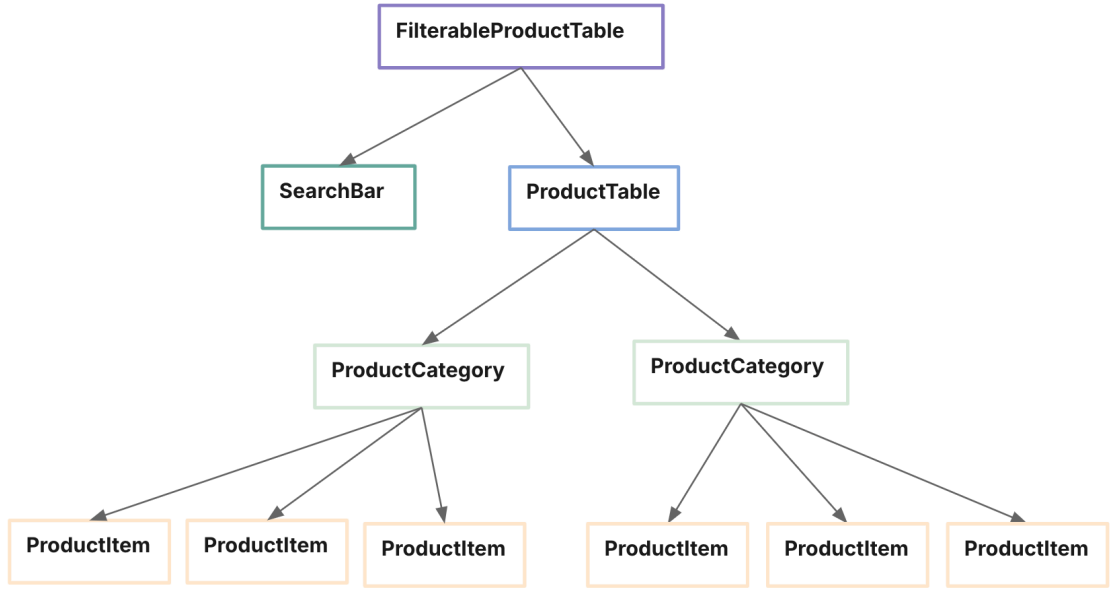
A row on screen does not prove a database write.

© 2026 Inspire | Confidential

Match page regions to a component tree



Visible regions



Parent and child components

Start with responsibilities, then read the code.

© 2026 Inspire | Confidential

Find the same parts in our Item list

Item list · teaching example

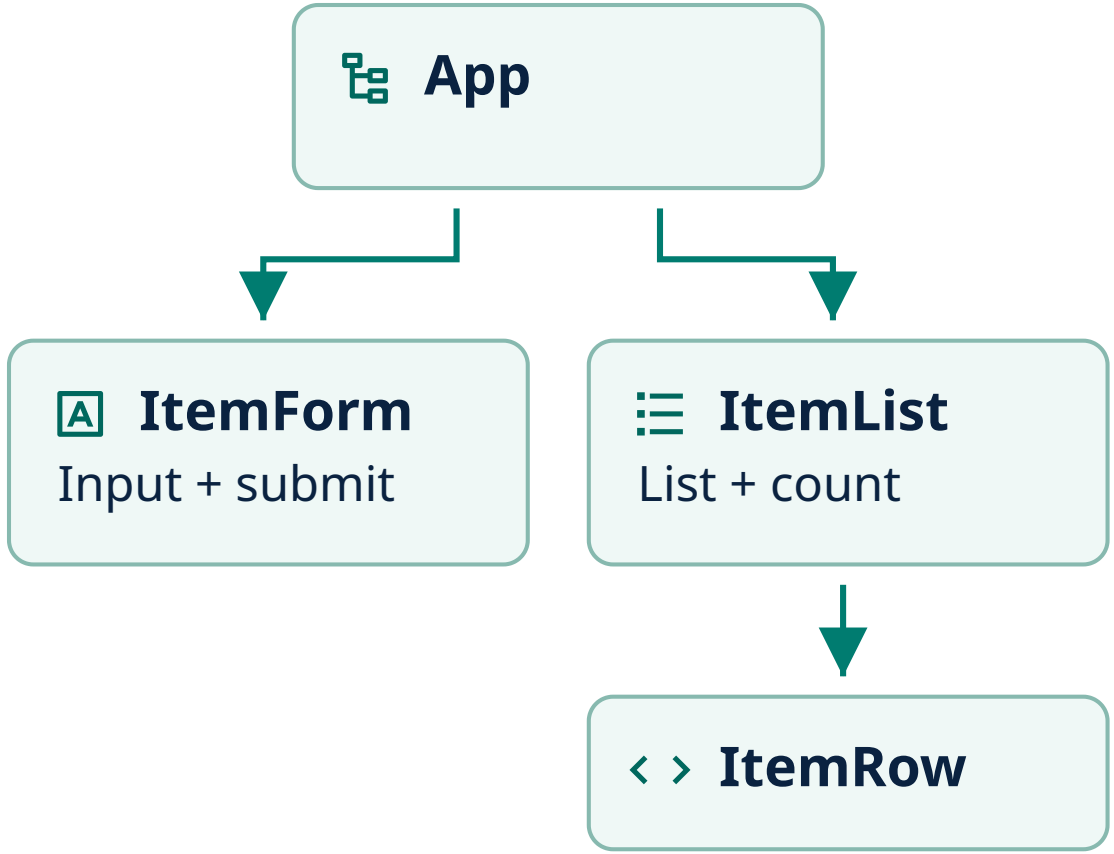
Label

Notebook

Add item

Items (1)

Notebook



App combines the form and list. ItemList renders one ItemRow per item.

© 2026 Inspire | Confidential

Describe the page for the current data

Direct DOM changes

Find each node. Change its text or children.

React components

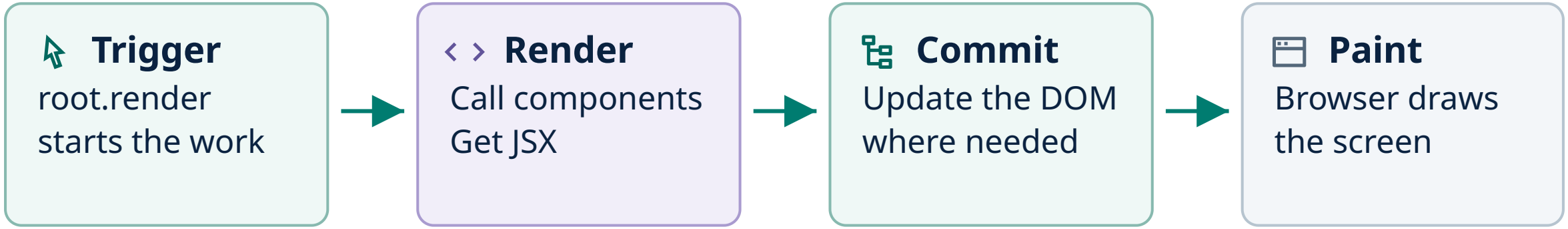
Describe the UI from data. React updates the DOM.

One list can drive both the rows and the count.

Components organize UI work. React is not the only way to build interaction.

© 2026 Inspire | Confidential

First render: code becomes a page



React prepares and commits the UI; the browser paints it.

Teaching example · JavaScript / JSX

```
const root = createRoot(document.getElementById('root'));
root.render(<App />);
```

JSX describes UI. Rendering and committing are separate steps.

© 2026 Inspire | Confidential

A component returns a UI description

Teaching example · JavaScript / JSX

```
function ItemRow(props) {
  return <li>{props.label}</li>;
}

<ItemRow label="Notebook" />
```

ItemRow names a component.

li names a built-in element.

{...} evaluates an expression.

JSX is transformed before it runs.

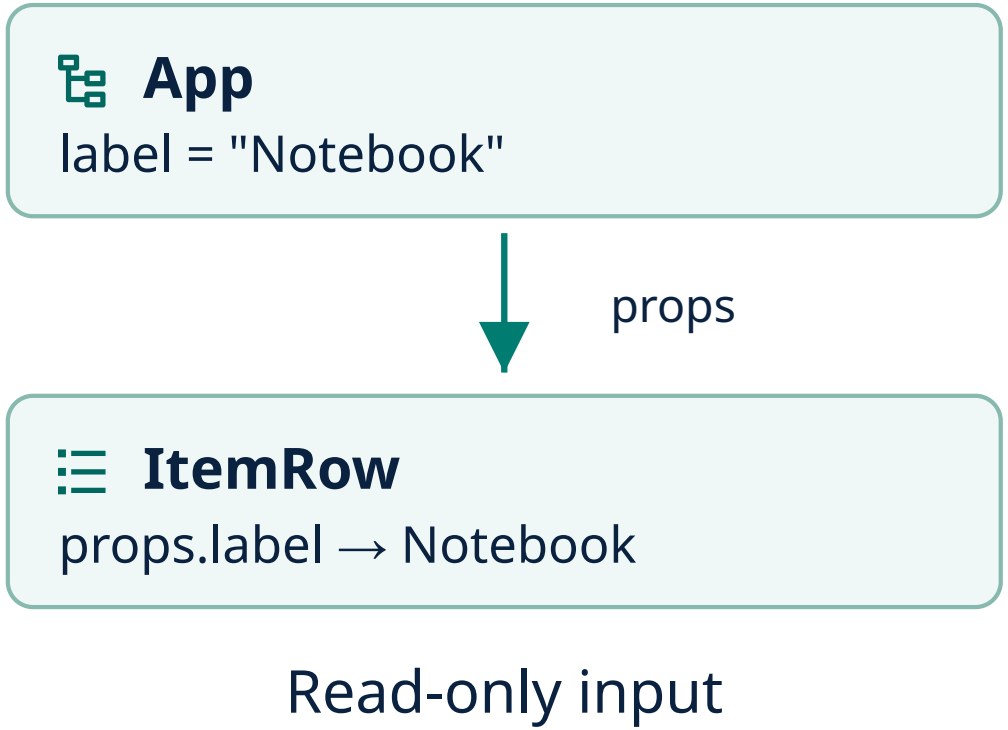
© 2026 Inspire | Confidential

Props carry values into a component

Teaching example · JavaScript / JSX

```jsx
function App() {
  return <ItemRow label="Notebook" />;
}

function ItemRow(props) {
  return <li>{props.label}</li>;
}
```



Read-only input

Parent chooses the value. Child reads it.

© 2026 Inspire | Confidential

An array supplies the rows

Teaching example · JavaScript / JSX

```
function ItemList(props) {
  return <ul>
    {props.items.map(item => (
      <ItemRow key={item.id}
        label={item.label} />
    ))}
  </ul>;
}
```

‹ › props.items

a1 · Notebook

b2 · Pencil

map in JSX

ul → ItemRow × 2

Notebook

Pencil

map returns descriptions. Put that array inside the returned JSX to display the rows.

Need objects, arrays or arrow functions? →

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Build a Small Result Display

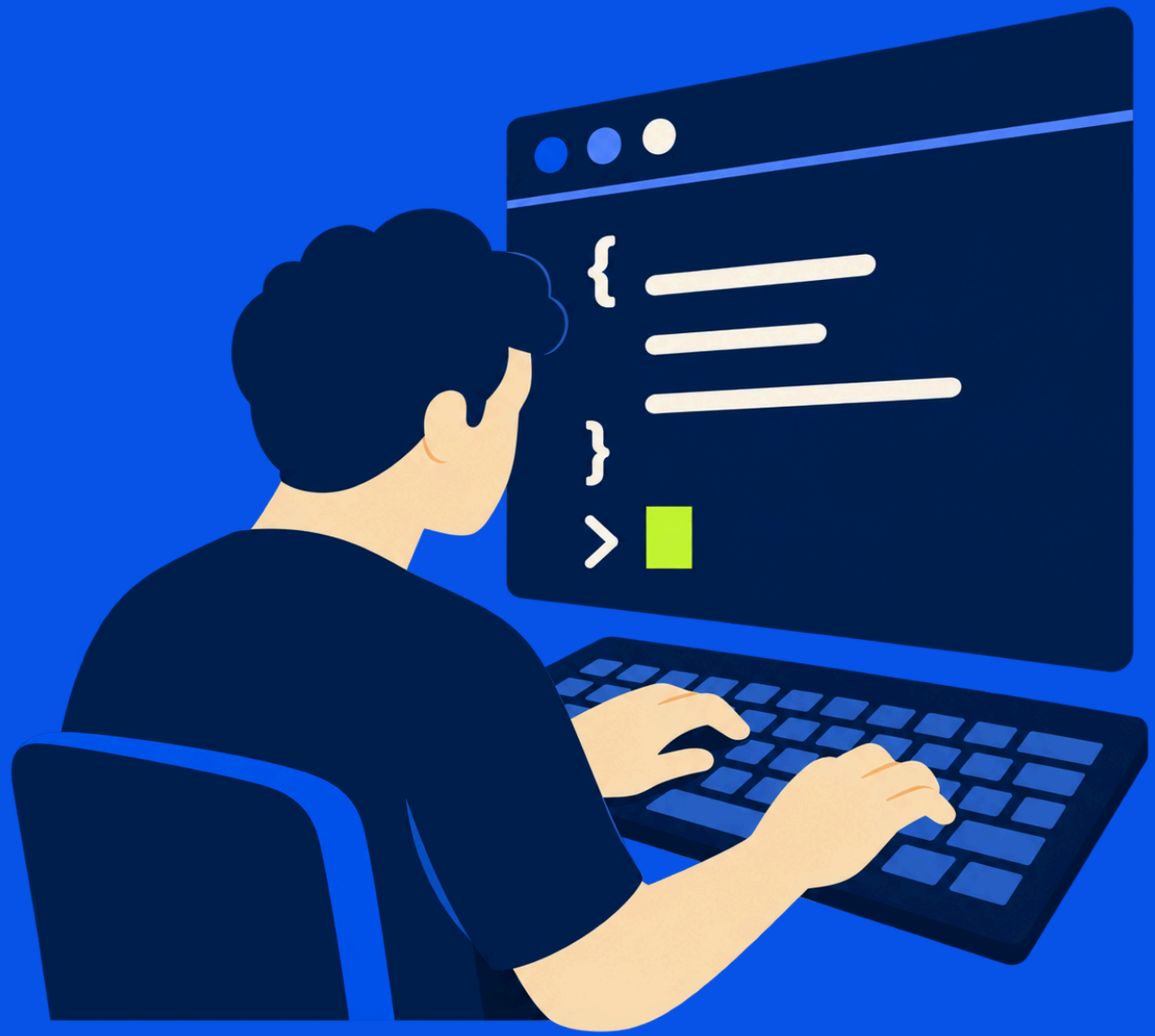


© 2026 Inspire | Confidential

INSPIRE AI

DEMO

Update State After a User Action



© 2026 Inspire | Confidential

Pass a function; call it on a click

Teaching example · JavaScript / JSX

```
function handleClick() {
  console.log("Clicked");
}

<button onClick={handleClick}>Try it</button>
```

handleClick

Pass the function value.

handleClick()

Call it now; use its result.

© 2026 Inspire | Confidential

State remembers a value between renders

Local variable

Teaching example · JavaScript / JSX

```
let count = 0;
function handleClick() {
  count = count + 1;
}
```

No React update is requested.

React state

Teaching example · JavaScript / JSX

```
const [count, setCount] =
  useState(0);
function handleClick() {
  setCount(count + 1);
}
```

The setter requests a new render.

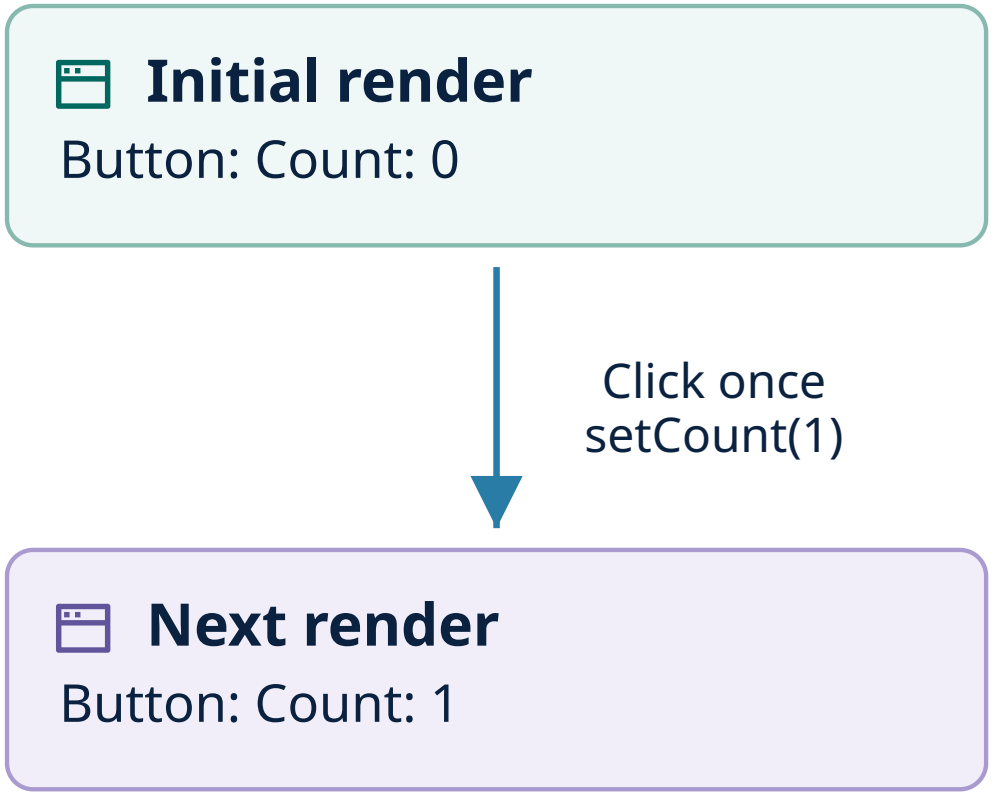
Fragments inside a component. Import useState from React; call Hooks at the top level.

© 2026 Inspire | Confidential

A click changes the displayed count

Teaching example · JavaScript / JSX

```jsx
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
  }
  return <button onClick={handleClick}>
    Count: {count}
  </button>;
}
```



Render <Counter /> in App. The button calls handleClick; JSX displays count.

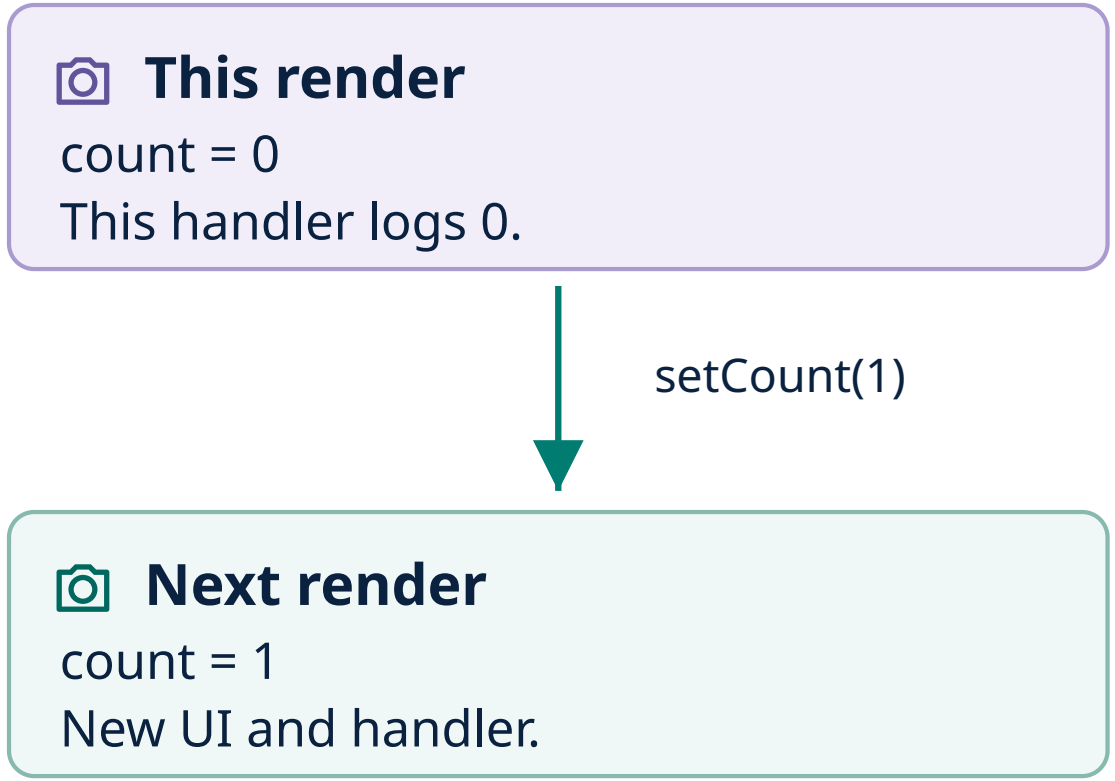
© 2026 Inspire | Confidential

A setter does not rewrite this render’s value

Teaching example · JavaScript / JSX

```
const [count, setCount] =
  useState(0);

function handleClick() {
  setCount(count + 1);
  console.log(count); // 0
}
```

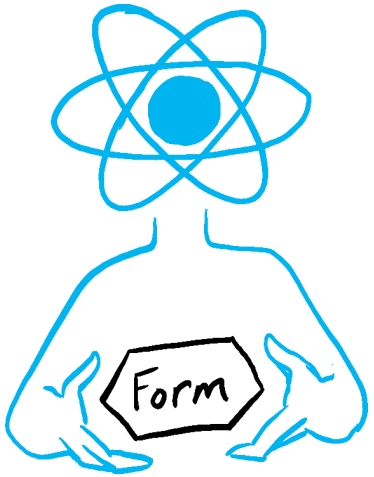


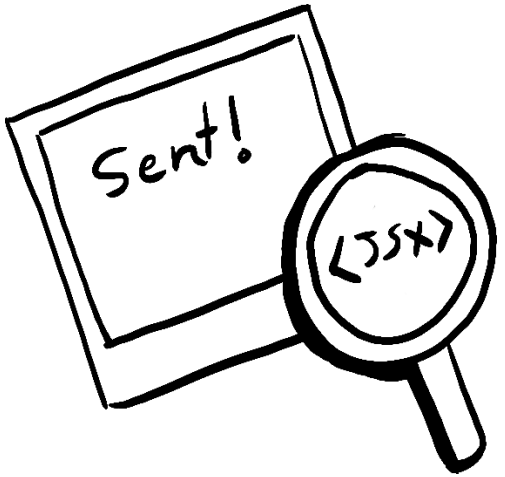
The first click keeps reading 0. The next render receives 1.

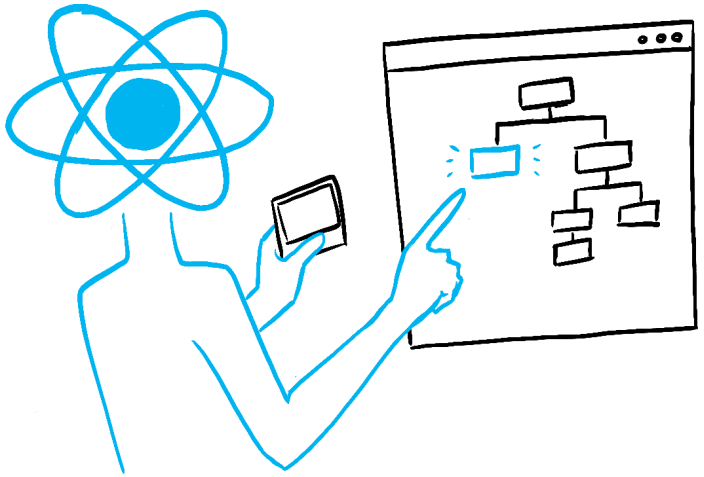
© 2026 Inspire | Confidential

An event starts the next render

Event handler → `setCount` requests a render







1 · Call the component

React runs the function.

2 · Describe the UI

The function returns JSX.

3 · Commit changes

React updates the DOM.

Illustrations: Rachel Lee Nabors · React documentation · CC BY 4.0 · Images unchanged; captions added.

The browser paints after the commit. Rerendering does not reload the page.

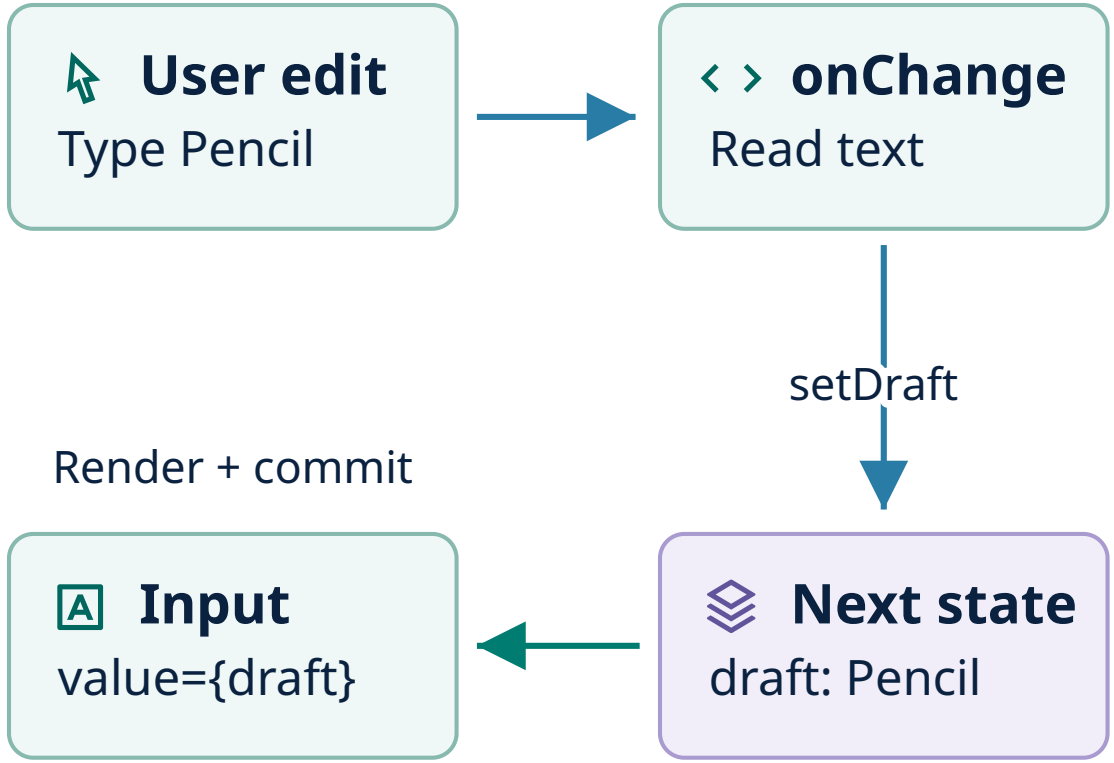
© 2026 Inspire | Confidential

The input displays state; typing updates it

Teaching example · JavaScript / JSX

```
const [draft, setDraft] =
  useState("");

<input value={draft}
  onChange={event =>
    setDraft(event.target.value)
  }
/>
```



A user edit triggers onChange. Updating value during rendering does not trigger it again.

© 2026 Inspire | Confidential

Typing edits. Submitting tries to add.

Inside ItemForm · handlers

```
function handleChange(event) {
  setDraft(event.target.value);
}

function handleSubmit(event) {
  event.preventDefault();
  // Validate, then call props.onAdd.
}
```

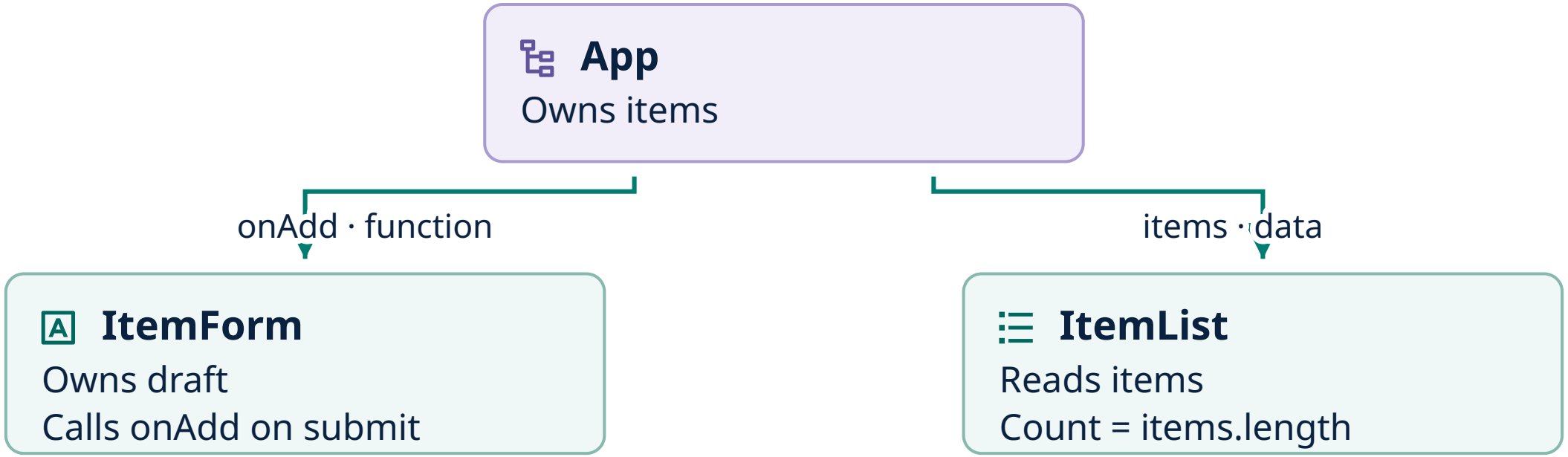
Inside ItemForm · returned JSX

```
<form onSubmit={handleSubmit}>
  <label htmlFor="label">
    Label
  </label>
  <input id="label" value={draft}
    onChange={handleChange} />
  <button type="submit">
    Add item
  </button>
</form>
```

handleChange names the previous inline handler. preventDefault prevents submit navigation.

© 2026 Inspire | Confidential

Keep shared data in a common parent



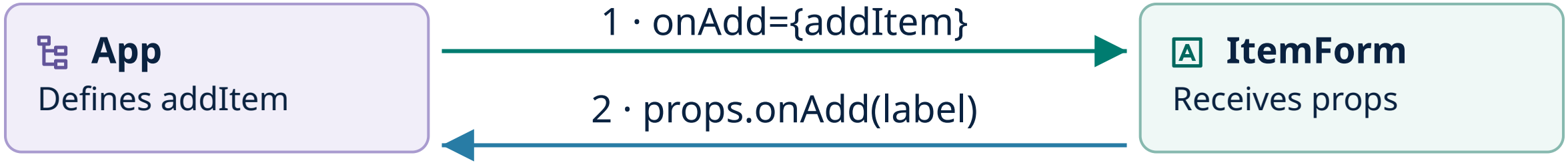
Teaching example · JavaScript / JSX

```
const [items, setItems] = useState([]);
```

Choose the nearest suitable common parent. Keep a form-only draft in the form.

© 2026 Inspire | Confidential

The child calls the function the parent passed



Inside App · pass the function

```
function addItem(label) {
  // Create the next items array.
}
return <ItemForm onAdd={addItem} />;
```

ItemForm outline · draft and JSX omitted

```
function ItemForm(props) {
  function handleSubmit(event) {
    event.preventDefault();
    props.onAdd(draft.trim());
  }
}
```

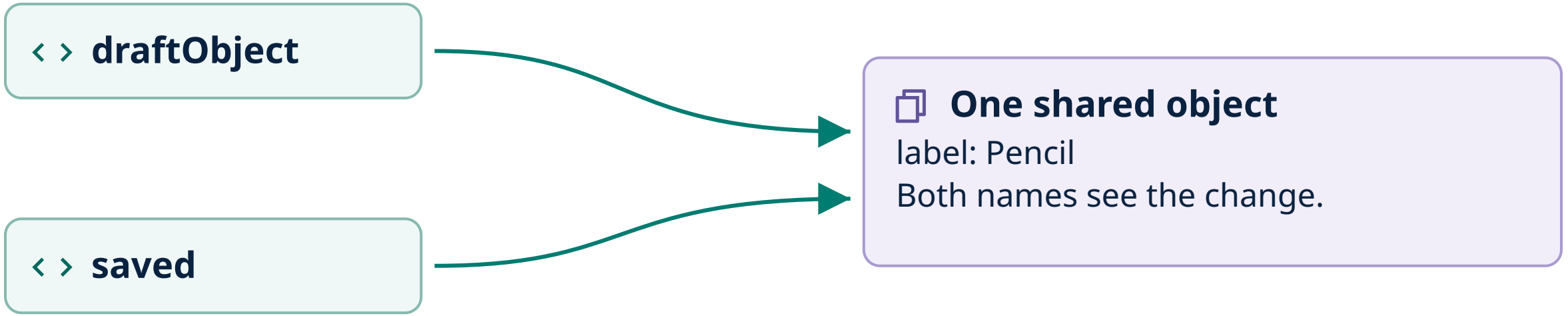
props.onAdd is addItem. Next: what data should addItem create?

© 2026 Inspire | Confidential

Assignment does not copy an object

Counterexample · if the draft were an object

```
const draftObject = { label: "Notebook" };
const saved = draftObject;
draftObject.label = "Pencil";
```



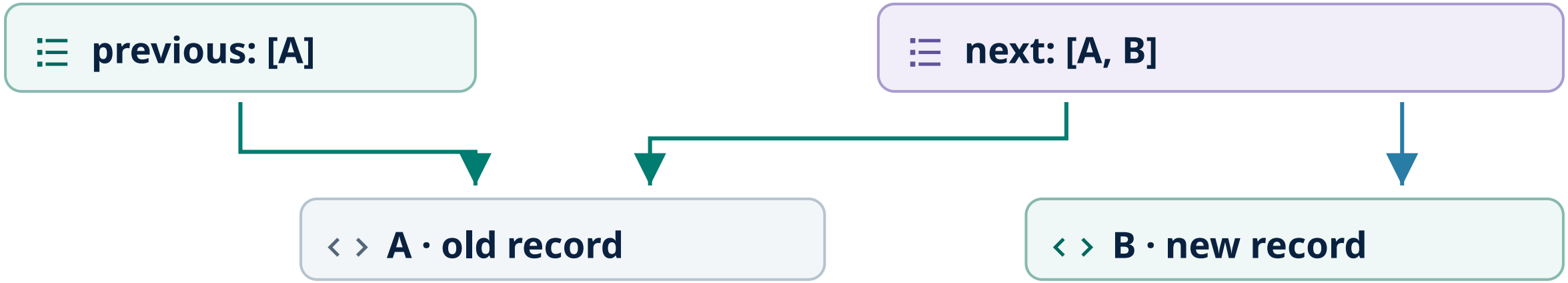
One shared object: editing the draft also changes the saved value. Our actual draft is text.

© 2026 Inspire | Confidential

Create a record, then a new array

Inside addItem · teaching example

```
const record = { id: crypto.randomUUID(), label };
const appendRecord = previous => [...previous, record];
setItems(appendRecord);
```



React calls appendRecord with the pending items state. It returns the next array.
previous is a parameter name. Old records stay unchanged; spread copies one level.

© 2026 Inspire | Confidential

New items reach the list through props

Inside App · after state and addItem

```
return <div>
  <ItemForm onAdd={addItem} />
  <ItemList items={items} />
</div>;
```

ItemList · count and rows from one array

```
function ItemList(props) {
  return <div>
    <h2>Items ({props.items.length})
    </h2>
    <ul>{props.items.map(item => (
      <ItemRow key={item.id}
        label={item.label} />
    ))}</ul>
  </div>;
}
```

Submit → props.onAdd → addItem → setItems.
Next render: App → new props.items → ItemList → rows and count.

© 2026 Inspire | Confidential

YOUR TURN

After one submit, what does editing change?

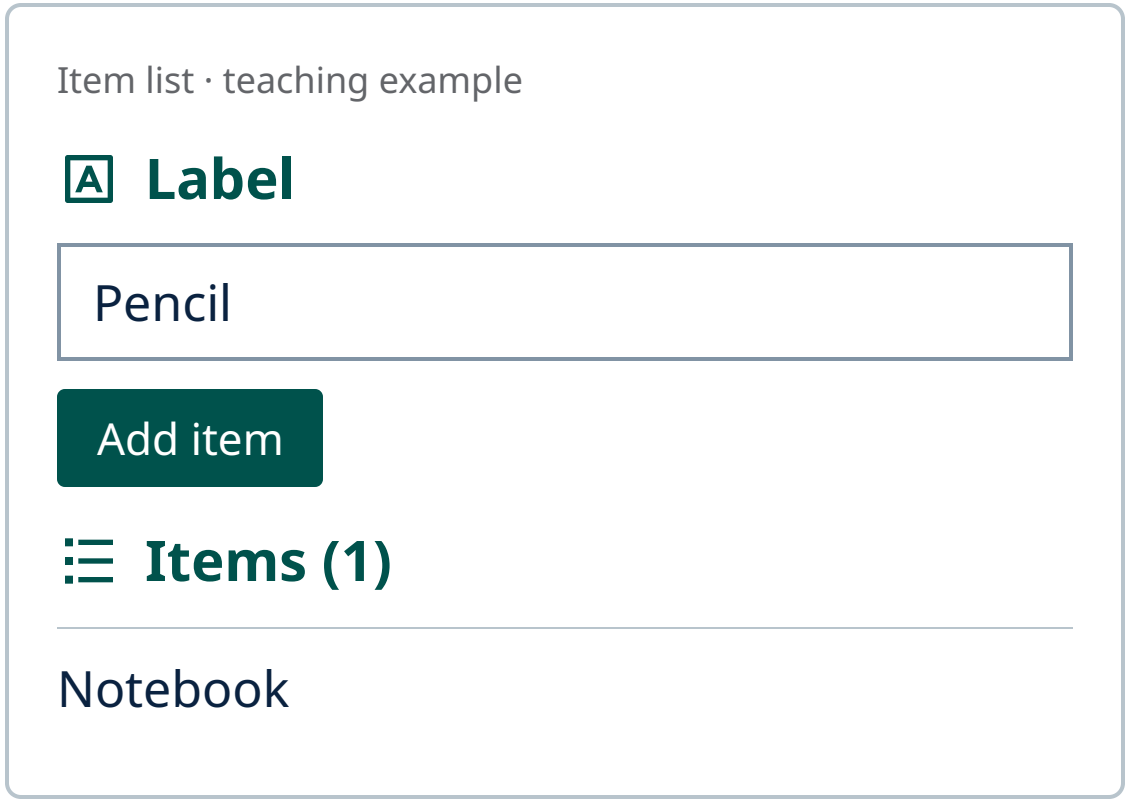
Predict the draft, the saved row and the count.

Explain which state owns each value.

Submit Notebook. Then type Pencil without submitting again.

© 2026 Inspire | Confidential

Editing keeps the submitted record unchanged



Expected behavior of the teaching example.

Draft: Pencil

Only ItemForm’s draft changed.

Saved row: Notebook

No second submit; no new record.

Count: 1 comes from items.length.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

PRACTICE

Connect the Form to Local State



© 2026 Inspire | Confidential

BREAK

Lunch



© 2026 Inspire | Confidential

MINI-GAME

Optional · A short group game



© 2026 Inspire | Confidential

PRACTICE

Correct one mismatch and name its state owner



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Explain One Event and State Update

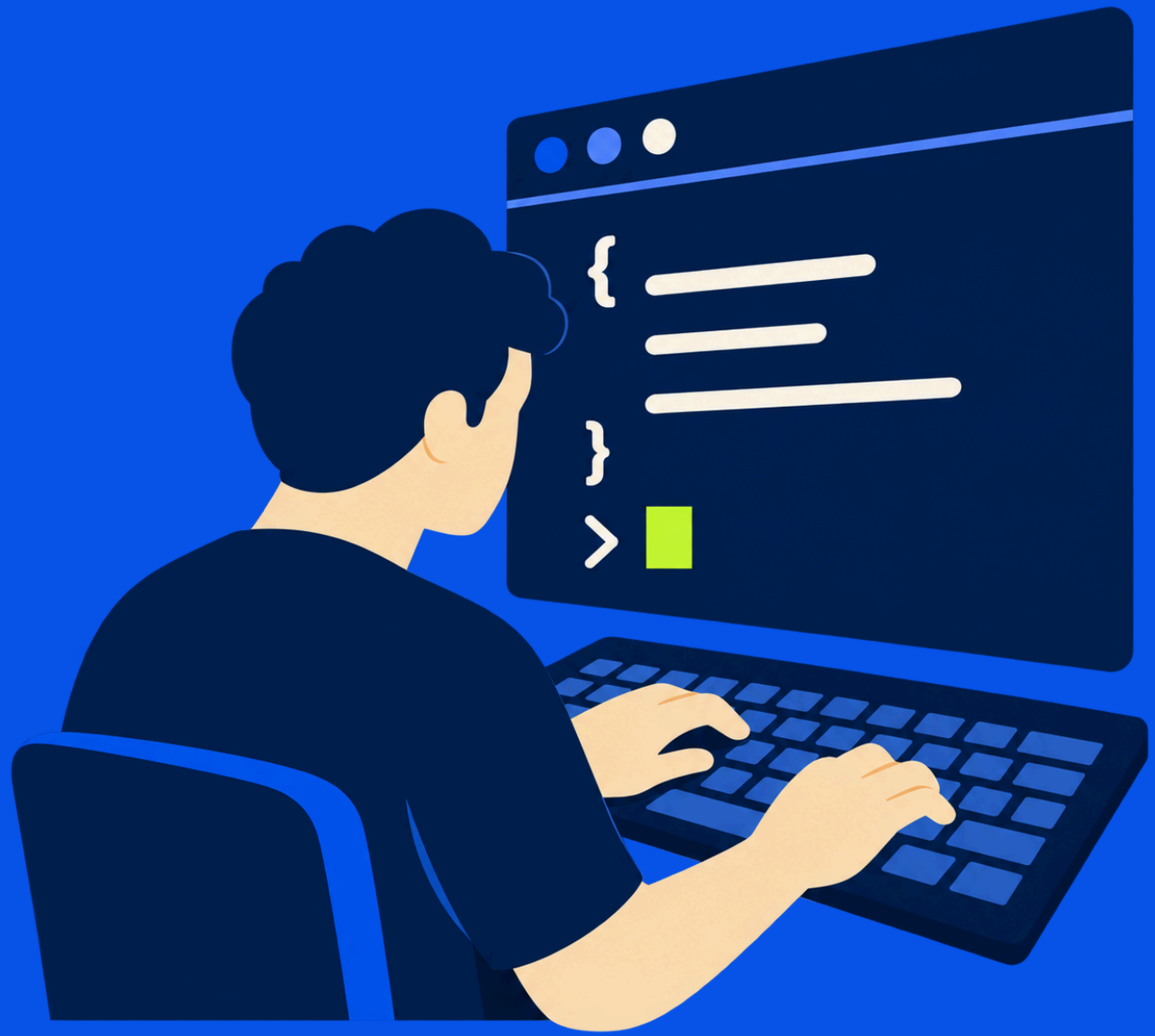


© 2026 Inspire | Confidential

INSPIRE AI

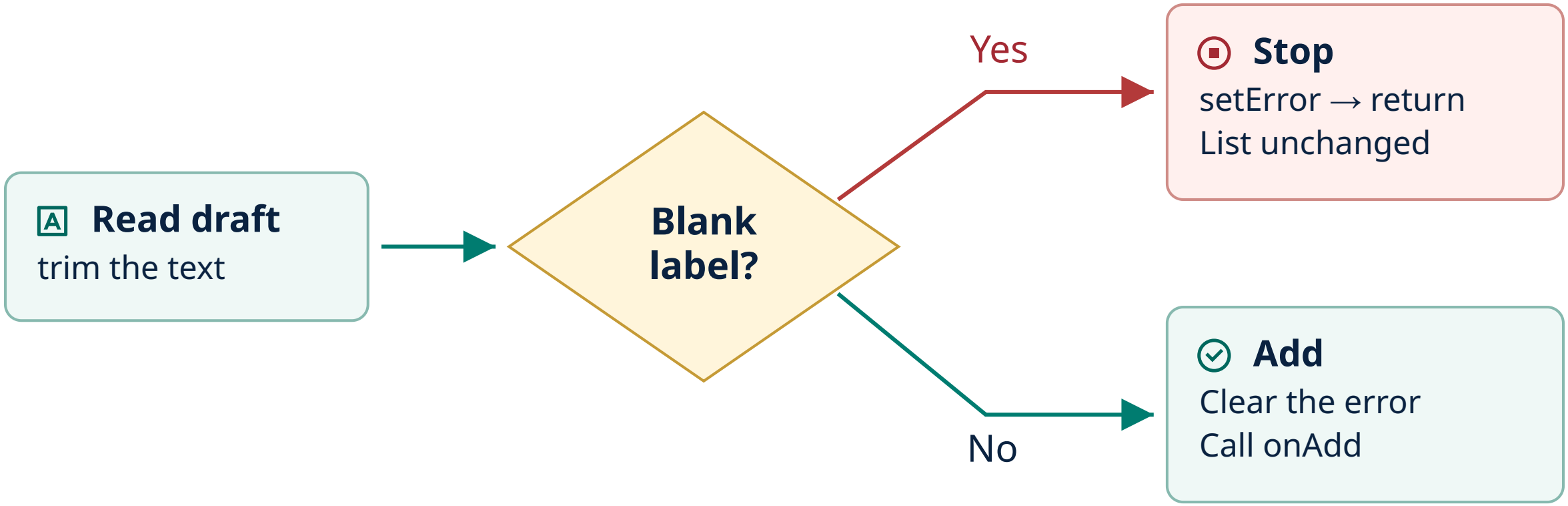
DEMO

Check Input and Show a Useful Message



© 2026 Inspire | Confidential

Validate before adding an item



Example rule: a label must not be blank. Use course rules for the parcel task.

© 2026 Inspire | Confidential

The failure branch never calls onAdd

Teaching example · JavaScript / JSX

```
function handleSubmit(event) {
  event.preventDefault();
  const label = draft.trim();
  if (!label) {
    setError("Enter a label.");
    return;
  }
  setError("");
  props.onAdd(label);
}
```

return exits this handler. The list changes only through props.onAdd.

© 2026 Inspire | Confidential

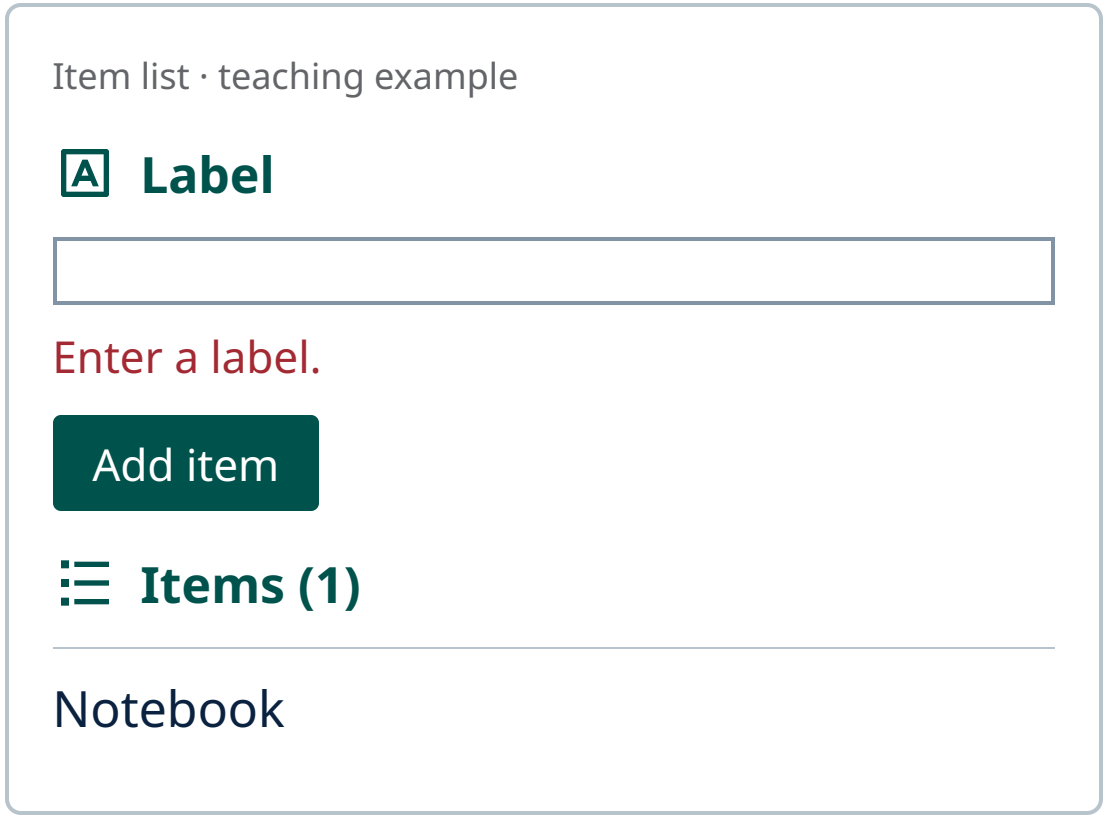
The form owns the draft and its message

Teaching example · JavaScript / JSX

```
const [error, setError] =
  useState("");

function handleChange(event) {
  setDraft(event.target.value);
  setError("");
}

{error && <p role="alert">
  {error}
</p>}
```



Blank submit: show error; keep the list. Edit: clear error. Submit again: recheck.

© 2026 Inspire | Confidential

Check invalid input and recovery

Action	Message	List
Submit a blank label	Enter a label.	Unchanged
Type Notebook	Cleared on edit	Unchanged
Submit Notebook	No error	One new record
Edit the draft again	No old error	Old records unchanged

Expected checks for the teaching example; not recorded execution results.

For numeric fields: text, empty input and conversion →

© 2026 Inspire | Confidential

React state lasts while the component is kept

Rerender

Same component position

`items: [Notebook]`



React keeps the state.

Reload

App starts again

`items: []`



Initial state returns.

No persistence in this example. Preset items do not prove your new item was saved.

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

PRACTICE

Add and Check an Input Error



© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

PRACTICE

Implement and Check a UI Change Independently



© 2026 Inspire | Confidential

Use the same path in the course UI

Find the value

Input, prop or state?

→

Trace its owner

Who may change it?

→

Show and check

Render the right field

Change one display. Explain where its value comes from.

Use the existing parcel starter and Curriculum checks. No price calculation or API call today.

Prerequisite help and optional topics →

© 2026 Inspire | Confidential

BREAK

Take a short break



© 2026 Inspire | Confidential

INSPIRE AI

SHOWCASE

Explain the UI Change and Check Results



© 2026 Inspire | Confidential

YOUR TURN

Why did the page show this result?

Trace the handler, state owner, props and next render.

Then explain what a page reload would change.

Choose one valid submit and one invalid submit.

© 2026 Inspire | Confidential

HOMEWORK

Display Zones and Delete Local Records



© 2026 Inspire | Confidential

Choose help by the missing explanation

REFERENCE · OPEN ONLY WHEN NEEDED

Required if unfamiliar

HTML and JSX

Functions, objects and arrays

Text and numeric input

Optional extensions

Context and Reducer

Router, Effects and storage

Not needed for the main example.

Skip a prerequisite explanation only when learners can already use it.

© 2026 Inspire | Confidential

Read the markup before editing it

PREREQUISITE HELP

Teaching example · JavaScript / JSX

```
<label htmlFor="label">Label</label>
<input id="label" className="field" />
<p>{draft}</p>
```

Tags name elements.
Attributes configure them.

htmlFor links label to input.
className selects a CSS class.

Return to page structure →

© 2026 Inspire | Confidential

Read values, functions and arrays

PREREQUISITE HELP

Teaching example · JavaScript / JSX

```
const item = { label: "Notebook" };
const readLabel = item => item.label;
const labels = [item].map(readLabel);
// labels: ["Notebook"]

const copy = { ...item };
const { label } = item; // same value as item.label
```

Arrow functions take arguments and return values. Spread makes a shallow copy.

Return to rendering a list → Return to appending a record →

© 2026 Inspire | Confidential

An input gives text, even for a number

PREREQUISITE HELP · BEFORE A NUMERIC-FIELD TASK

Teaching example · JavaScript / JSX

```
const text = weightText.trim();
if (text === "") { /* missing input */ }
else {
  const weight = Number(text);
  if (!Number.isFinite(weight)) { /* invalid number */ }
  else { /* apply the specified range rule */ }
}
```

Number("") is 0. Check missing text before conversion.

Return to input checks →

© 2026 Inspire | Confidential

Extra tools address extra state needs

OPTIONAL · MAIN PATH DOES NOT DEPEND ON THESE TOOLS

Context

Read shared data across distant components without passing it through every level.

Reducer

Describe related state transitions in one update function.

Keep ownership clear whichever tool you choose.

Return to the course task →

© 2026 Inspire | Confidential

Add a tool when its job appears

OPTIONAL · PREVIEW OF LATER NEEDS

Need	Possible tool
Match a URL to a view	Router
Use ready-made controls	Component library
Synchronize with an external system	Effect / framework data tools
Keep data after reload	Browser / server storage

Compute derived values during render. Handle a submit in its event handler.

Return to the course task →

© 2026 Inspire | Confidential